

LAB – JAVA REMOTE METHOD INVOCATION (RMI)

Student Details:

- **Name:** Mary Queenvine Muthoni
- **Admission Number:** 158716
- **Unit:** Distributed Systems
- **Title:** String Concatenation Using Java RMI
- **Group Number:** 14

Theory

In a distributed system, processes often need to communicate and invoke operations on objects located in completely different memory spaces or on entirely different physical machines. Java Remote Method Invocation (RMI) is an API that provides a mechanism to allow a Java program to invoke methods on an object running in another Java Virtual Machine (JVM).

To achieve this, RMI relies on a few core components: a remote interface that defines the methods available to remote clients, a servant object that actually implements those methods, a host server that registers the object, and a client that looks up the object and calls its methods. An RMI Registry acts as the central directory service, allowing the server to bind the remote object to a specific name and the client to discover it using that same name.

Aim

To implement and demonstrate a distributed client-server architecture using Java RMI, observing how a client application can pass two independent string variables over a local network to a remote server, which then concatenates the strings and returns the combined result.

Algorithm

1. Define a remote interface (ServerInterface) that extends `java.rmi.Remote` and declares a `concat` method that throws a `RemoteException`.
2. Create a servant class (Servant) that implements the remote interface and provides the logic to concatenate two strings.
3. Create a server application (Server) that instantiates the servant object.
4. The server registers this servant object with the RMI registry using a unique binding name ("Server").
5. Create a client application (Client) that connects to the RMI registry using the server's network address.
6. The client looks up the registered remote object by its bound name.
7. The client captures user input for two strings, invokes the remote `concat` method, and displays the concatenated string returned by the server.

Implementation

The lab was implemented using Java in a Linux environment running the Zsh shell. The system utilizes four separate files to establish the RMI framework:

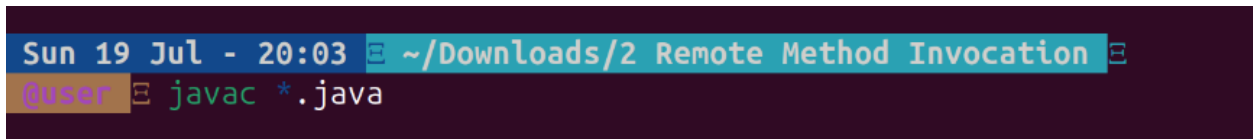
- ServerInterface.java - defines the remote method signature.
- Servant.java - contains the execution logic to return $a + b$.
- Server.java - binds the remote object to the RMI registry.
- Client.java - handles user input and method invocation.

(a) Software Requirements

- Java Development Kit (JDK) installed via apt
- Java Compiler (javac)
- Java Runtime (java)
- RMI Registry tool (rmiregistry)
- Linux Terminal (Zsh)

(b) Compilation

The source files were compiled from the lab directory using the Java compiler.

A terminal window with a dark purple background. The title bar is light blue and contains the text "Sun 19 Jul - 20:03" followed by a window icon, "~/Downloads/2 Remote Method Invocation", and another window icon. The terminal prompt is "@user". The command "javac *.java" has been entered and is highlighted in green.

This produced the executable bytecode .class files alongside the .java source files.

(c) Execution - Starting the Registry

Before the server could bind the remote object, the RMI registry was initialized in a new terminal tab. The registry successfully started, throwing standard security manager deprecation warnings expected in modern Java environments.

```
Sun 19 Jul - 19:50 ~ /Downloads/2 Remote Method Invocation
@user rmiregistry
WARNING: A terminally deprecated method in java.lang.System has been called
WARNING: System::setSecurityManager has been called by sun.rmi.registry.RegistryImpl
WARNING: Please consider reporting this to the maintainers of sun.rmi.registry.RegistryImpl
WARNING: System::setSecurityManager will be removed in a future release
```

(d) Execution - Server Deployment

The server application was launched in a third terminal window to host the Servant object. To prevent connection refusal errors caused by local network bindings, the server was explicitly bound to the loopback address using the `java.rmi.server.hostname` property.

```
Sun 19 Jul - 20:10 ~ /Downloads/2 Remote Method Invocation
user java Server
```

The server process remained active in the background, listening for incoming RPC (Remote Procedure Call) requests.

(e) Execution - Running the Client In a fourth terminal window, the client application was executed. The server address was set to the local loopback address (127.0.0.1), and two strings were passed to the remote object.

```
Sun 19 Jul - 19:50 ~ /Downloads/2 Remote Method Invocation
@user java Client
Enter the Server address :
127.0.0.1
Enter first string :
Mary
Enter second string :
Queenvine
Concatenated String : MaryQueenvine
```

Conclusion

This lab successfully demonstrated the mechanics of Java RMI by decoupling method execution from method invocation. By navigating common network binding issues and following the correct startup sequence, compiling the code, initializing the registry, deploying the server, and running the client, the application proved that distributed network processes can seamlessly share

data and processing workloads. The successful concatenation of the two strings verified that the remote object binding and client lookup procedures functioned exactly as intended over the local network.